

Good evening, DJ GABRU. Welcome to Notion.

I'm Notion AI. Yes, another AI, I know. But I'll get work done for you and build you anything.

Let's get started. Tell me what you're here to do, or jump in with what most people start with.

GPT-5.6 Sol Code Review Prompt

You are a senior quantitative trading systems engineer. I need you to do a deep, critical code review of a live crypto trading engine called Engine 1. I have pasted the full source code for the main engine, the backtest setup, and the strategy base below.

System Context

Engine 1 is a Python-based live trading signal generator for crypto perpetual futures. Here is how it works end-to-end:

Data ingestion: A Playwright browser scrapes Coinglass (a crypto derivatives analytics platform) at regular intervals. It extracts real-time per-symbol data including: funding rate, open interest, long/short ratio, aggregated liquidations (long and short), CVD (cumulative volume delta from futures and spot), RSI, and footprint/orderflow data (coins bid/ask, dollar bid/ask, whale index, trade counts).

Candle building: Each scrape tick updates a live rolling candle (15-min OHLCV + all above indicators). Completed candles are pushed into a per-symbol deque (maxlen=1200 bars, ~300 hours of 15-min data).

Feature engineering (compute_features): Builds a feature matrix from the candle deque using the prepup function from unified_backtest.py. Key features: suu-bar EMA macro trend (macro = +1 bull / -1 bear / 0 neutral), ATR, z-scored CVD at windows 4/10/20, BTC cross-asset CVD z-scores, OI z-score, L/S ratio z-score, funding rate, rolling liquidation sums, vol regime z-score.

Prediction: Two LightGBM boosters per symbol (long model, short model) run on the latest feature row. They output probabilities p_long and p_short.

Signal gating: A signal fires only when ALL of the following are true:

- p_long >= confidence_threshold AND macro == 1 (for LONG)
- p_short >= confidence_threshold AND macro == -1 (for SHORT)
- ATR% is below a vol_limit cap
- No existing active trade for this symbol
- Fewer than 4 concurrent positions open

Trade tracking (Engine1TradeTracker): Logs entry/exit with Zeno risk sizing (0.5% of current capital per trade). Saves to its own isolated JSON log file (Engine_1_trade_logs.json). Displays a live Rich terminal panel showing active trades, history, capital, PnL, and winrate.

Terminal UI: Uses rich.live.Live with a multi-panel layout showing: per-symbol live data table (price, funding, OI, CVD, LS ratio, liquidations, RSI, ARM status), and a bottom trade stats panel.

Seeding: On startup, the engine loads 2,000+ historical candles per symbol from an Excel workbook (Engine_1/Seeding/combined_seed_history.xlsx) into the deque, giving the 800-bar EMA a proper warm start before any live tick arrives.

Known Recent Fixes Applied

- Silent predict() exception swallowed — now caught and logged with [Strategy] symbol prediction error: e
- Excel seeding rows had string open_time (e.g. "1720000000 IST") — IST suffix now stripped and correctly converted to integer Unix timestamps to fix deque sort order
- TradeTracker previously loaded from Engine 3's legacy simulation logs (corrupted PnL in the $\sqrt{x}$ Invalid equation $\wedge\{+-\}\%0.0+\d*\%?$ $\sqrt{x}$ Invalid equation (pnl_usd:2f) (+ (pnl_pct:+2f)%/f) if pnl_usd >= 0 else f"[bold red]: $\sqrt{x}$ Invalid equation (pnl_usd:2f) (+ (pnl_pct:+2f)%/f) if pnl_usd >= 0 else f"[bold red]: \$(abs(pnl_usd):2f) ((pnl_pct:+2f)%/f)" reason = tr.get('exit_reason', 'EXIT') history_lines.append(f"[tr{'symbol'}] | [dir_str] | Exit: (tr['exit_price']:.4f) | Reason: (reason) | Final: (pnl_str)")

history_text = "\n".join(history_lines) if history_lines else "[dim]No trade history[/dim]"

Stats string
winrate = stats['winrate']
total_pnl = stats['total_pnl_usd']
pnl_pct = total_pnl / 5000.0 * 100.0
pnl_clr = "green" if total_pnl >= 0 else "red"
pnl_sign = "+" if total_pnl >= 0 else ""

stats_text = (
 f"Initial Capital: [bold]\$5,000.00[/bold] | Current Capital: [bold]\$ {stats['current_capital']:.2f}/f | "
 f"[Total PnL: [bold] (pnl_clr){pnl_sign}\${total_pnl:2f} ((pnl_pct+2f)%/f) | "
 f"Trades: [bold]{stats['total']}/f | Winrate: [bold]{winrate:1f}%/f"
)

trade_table = Table(show_header=True, header_style="bold bright_magenta",
border_style="magenta", expand=True)
trade_table.add_column("Active Trades", justify="left", ratio=1)
trade_table.add_column(stats_text, justify="left", ratio=1)
trade_table.add_row(active_text, history_text)

return Group(t, trade_table)

async def renderer_loop(store: SnapshotStore, stop: asyncio.Event) -> None:
 console = Console()
 with Live(render_table(store.snapshot(), store.trade_tracker), console=console,
refresh_per_second=REFRESH_HZ, screen=False) as live:
 while not stop.is_set():
 snap = store.snapshot()

```

live.update(render_table(snap, store.trade_tracker))
try:
    serializable_snap = {}
    for sym, a in snap.items():
        serializable_snap[sym] = {
            "price": a.price, "volume": a.volume, "oi": a.oi, "rsi": a.rsi, "fut_cvd": a.fut_cvd, "spot_cvd":
            a.spot_cvd,

            "liq_long": a.liq_long, "liq_short": a.liq_short, "funding": a.funding,
            "ls_ratio": a.ls_ratio, "oi": a.oi,
            "coins_bid": a.coins_bid, "coins_ask": a.coins_ask,
            "dollars_bid": a.dollars_bid, "dollars_ask": a.dollars_ask,
            "whale_idx": a.whale_idx, "tk_buy_cnt": a.tk_buy_cnt, "tk_sell_cnt": a.tk_sell_cnt,
            "fp_delta": a.fp_delta, "fp_poc": a.fp_poc,
            "strategy_armed": a.strategy_armed, "ts_ns": a.ts_ns
        }
    with open("Engine_1/Seeding/snapshot_debug.json", "w", encoding="utf-8") as f:
        json.dump(serializable_snap, f, indent=4)
    except Exception:
        pass
    await asyncio.sleep(1.0 / REFRESH_HZ)

# --- WATCHDOG ---
async def watchdog(components: List[Any], stop: asyncio.Event) -> None:
    while not stop.is_set():
        now = time.time_ns()
        for c in components:
            if hasattr(c, 'last_heartbeat_ns') and now - c.last_heartbeat_ns > 15_000_000_000:
                print(f"[Watchdog] [WARN] Subsystem '{c.class.name}' hung. Heartbeat silent >15s.")
                await asyncio.sleep(3.0)

def combine_seeding_files() -> None:
    import glob
    import copy
    from openpyxl import load_workbook, Workbook
    from openpyxl.utils import get_column_letter

    files = glob.glob("Engine_1/Seeding/*_seed_history.xlsx")
    files = [f for f in files if "combined_seed" not in os.path.basename(f).lower()]
    if not files:
        print(f"[Setup] No seeding files found to combine.")
        return

    print(f"[Setup] Combining {len(files)} seeding files into a single workbook...")
    combined_wb = Workbook()
    default_sheet = combined_wb.active
    combined_wb.remove(default_sheet)

    for f in sorted(files):
        symbol = os.path.basename(f).replace("seed_history.xlsx", "")
        try:
            wb = load_workbook(f)
            source_ws = wb.active
            target_ws = combined_wb.create_sheet(title=symbol[:31])

            for row in source_ws.iter_rows():
                for cell in row:
                    new_cell = target_ws.cell(row=cell.row, column=cell.column, value=cell.value)
                    if cell.has_style:
                        new_cell.font = copy.copy(cell.font)
                        new_cell.fill = copy.copy(cell.fill)
                        new_cell.border = copy.copy(cell.border)
                        new_cell.alignment = copy.copy(cell.alignment)
                        new_cell.number_format = cell.number_format

            for col in source_ws.columns:
                col_letter = get_column_letter(col[0].column)
                target_ws.column_dimensions[col_letter].width =
                source_ws.column_dimensions[col_letter].width
            wb.close()
        except Exception as copy_exc:
            print(f"[Setup] [WARN] Failed to copy {symbol} sheet: {copy_exc}")

    combined_filename = "Engine_1/Seeding/combined_seed_history.xlsx"
    try:
        combined_wb.save(combined_filename)
        print(f"[Setup] Combined workbook saved successfully: {combined_filename}")

    # Clean up individual seed files
    for f in files:
        try:
            os.remove(f)
        except:
            pass
    print(f"[Setup] Cleaned up individual symbol seeding files.")
    except Exception as e:
        print(f"[Setup] [WARN] Failed to save combined workbook: {e}")

# --- MAIN CONTROLLER ---
async def main(skip_seed: bool = False) -> None:
    # 0. Live Model Retraining on latest Parquet data
    print(f"[Setup] Running Live Model Retraining on latest Parquet data...")
    try:
        sys.path.append(os.path.join(os.getcwd(), "Engine_1", "alpha_squeezer_v17"))
        from model_trainer import train_models
        train_models()
    except Exception as retrain_err:
        print(f"[Setup] [WARN] Failed to retrain models: {retrain_err}")

    # Initialize LiveStrategyPredictor & load cached history
    predictor = LiveStrategyPredictor(ALL_SYMBOLS)
    predictor.load_history_from_disk()

    trade_tracker = Engine1TradeTracker()
    store = SnapshotStore(ALL_SYMBOLS, predictor, trade_tracker)
    stop = asyncio.Event()

```

```

print("[Setup] Launching Chromium instance with persistent profile...")
async with async_playwright() as pw:
    user_data_dir = os.path.join(os.getcwd(), "chrome_profile")
    ctx = await pw.chromium.launch_persistent_context(
        user_data_dir,
        headless=False,
        viewport={"width": 1920, "height": 1080},
        args=[
            "--disable-features=CalculateNativeWinOcclusion",
            "--disable-background-timer-throttling",
            "--start-maximized",
            "--remote-debugging-port=9222"
        ]
    )

# 1. Performing Session Login first
print("[Setup] Navigating to Coinglass Login...")
login_page = await ctx.new_page()

for attempt in range(3):
    try:
        await login_page.goto("https://www.coinglass.com/login", wait_until="load",
            timeout=45000)
        break
    except Exception as exc:
        print(f"[Setup] [WARN] Login navigation attempt {attempt+1} failed: {exc}")
        if attempt == 2:
            raise exc
        await asyncio.sleep(5.0)
        await asyncio.sleep(5)

os.makedirs("Engine_1/Seeding", exist_ok=True)
await login_page.screenshot(path="Engine_1/Seeding/login_init.png")

email_input = login_page.locator("input[placeholder='Email']").first
if await email_input.count() > 0:
    print("[Setup] Inputting credentials...")
    await email_input.click()
    await email_input.fill("singhkarabir0248@gmail.com")
    await asyncio.sleep(0.3)

    pass_input = login_page.locator("input[placeholder='Password']").first
    await pass_input.click()
    await pass_input.fill("Lu$er2hero")
    await asyncio.sleep(0.3)

    await login_page.screenshot(path="Engine_1/Seeding/login_filled.png")
    print("[Setup] Submitting login form...")

    # Try explicit button click first, fallback to JS click, last resort Enter key
    try:
        btn = login_page.locator("button:has-text('Login')").first
        if await btn.count() > 0:
            await btn.wait_for(state="visible", timeout=5000)
            await btn.click()
        else:
            raise Exception("button not found via locator")
    except Exception:
        try:
            await login_page.evaluate("""() => {
                const b = Array.from(document.querySelectorAll("button"))
                .find(ei => ei.textContent.trim() === 'Login');
                if (b) b.click();
            }""")
        except Exception:
            # Most reliable: press Enter on password field
            await pass_input.press("Enter")

print("[Setup] Waiting for post-login redirect...")
try:
    await login_page.wait_for_url(lambda url: "/login" not in url, timeout=20000)
    print("[Setup] Login successful — redirected away from /login.")
except Exception:
    print("[Setup] [WARN] No redirect detected — may already be logged in or login failed.")
    await login_page.screenshot(path="Engine_1/Seeding/login_after_submit.png")
    print("[Setup] Waiting 5 seconds to ensure session cookies are fully persisted...")
    await asyncio.sleep(5.0)
    else:
        print("[Setup] Form inputs not detected, assuming session already active.")

# 2. Open Scraping Tabs (while login page is open to hold session cache)
tab1 = CoinglassTab(ctx, TAB1_SYMBOLS, store, "TAB_1")
tab2 = CoinglassTab(ctx, TAB2_SYMBOLS, store, "TAB_2")
binance = BinanceFootprintFeed(ALL_SYMBOLS, store)

await asyncio.gather(tab1.start(), tab2.start(), binance.start())

# Close login page now that layout tabs have initialized
try:
    await login_page.close()
except:
    pass

# 3. Configure grid symbols & indicators
focus_lock = asyncio.Lock()
await asyncio.gather(
    tab1.inject_and_configure_all(focus_lock),
    tab2.inject_and_configure_all(focus_lock)
)

# 4. Historical Seeding
from concurrent.futures import ThreadPoolExecutor
excel_pool = ThreadPoolExecutor(max_workers=4)

```

```

if skip_seed:
    print("[Setup] --skip-seed flag active. Skipping historical seeding.")
else:
    async def seed_wrapper(tab: CoinglassTab, sym: str):
        await tab.seed_symbol(sym, excel_pool, focus_lock)

    seeding_tasks = [seed_wrapper(tab1, sym) for sym in TAB1_SYMBOLS] + \
        [seed_wrapper(tab2, sym) for sym in TAB2_SYMBOLS]

    print("[Setup] Launching historical seeding...")
    await asyncio.gather(*seeding_tasks)
    print("[Setup] Seeding phase complete! Starting real-time feeds...")
    combine_seeding_files()

# 5. Run Live feeds & Terminal display
tasks = [
    asyncio.create_task(tab1.poll_loop()),
    asyncio.create_task(tab2.poll_loop()),
    asyncio.create_task(binance.run()),
    asyncio.create_task(renderer_loop(store, stop)),
    asyncio.create_task(watchdog([tab1, tab2, binance], stop))
]

# Handle graceful exit triggers
loop = asyncio.get_running_loop()
def sig_handler():
    print("\n[Exit] Termination signal received. Stopping...")
    stop.set()
    tab1.running = False
    tab2.running = False
    binance.running = False

for sig in (signal.SIGINT, signal.SIGTERM):
    try:
        loop.add_signal_handler(sig, sig_handler)
    except NotImplementedError:
        pass

try:
    while not stop.is_set():
        await asyncio.sleep(1.0)
except (KeyboardInterrupt, SystemExit):
    sig_handler()

for t in tasks:
    t.cancel()
await asyncio.gather(*tasks, return_exceptions=True)

excel_pool.shutdown(wait=True)
await ctx.close()

print("[Exit] Shutdown complete.")

if name == "main":
    import argparse
    parser = argparse.ArgumentParser(description="Coinglass + Binance Footprint Scraper")
    parser.add_argument("--skip-seed", action="store_true", help="Skip historical Excel seeding and go straight to live feeds")
    args = parser.parse_args()
    asyncio.run(main(skip_seed=args.skip_seed))

## 2. Engine 1/alpha_squeezer_v17/unified_backtest.py python
import importlib.machinery
import importlib.util
import os
import sys
import pandas as pd
import numpy as np

# Get the path to the compiled .pyc file in the same directory
dir = os.path.dirname(os.path.abspath(file))
pyc_path = os.path.join(dir, "unified_backtest.pyc")

# Load the compiled module under an internal name to prevent recursion
loader = importlib.machinery.SourcelessFileLoader("unified_backtest_compiled",
    pyc_path)
spec = importlib.util.spec_from_loader("unified_backtest_compiled", loader)
compiled_module = importlib.util.module_from_spec(spec)
sys.modules["unified_backtest_compiled"] = compiled_module
spec.loader.exec_module(compiled_module)

# Expose all attributes to this module's namespace
globals().update(compiled_module.dict)

# Apply directory path override to point to the correct Google Drive folder
PQ_DIR = r"G:\My Drive\Trading_Data\15m\parquet"
compiled_module.PQ_DIR = PQ_DIR

# --- CUSTOM REPLACEMENT LOADER & FEATURE ENGINEERING OVERRIDES ---

def zscore(s, w):
    return (s - s.rolling(w, min_periods=1).mean()) / s.rolling(w,
        min_periods=1).std().replace(0, 1e-10)

def custom_load_asset(symbol):
    # Try the default G Drive pattern
    summary_path = os.path.join(PQ_DIR,
        f"Master({symbol})15m_Final_Summary.parquet")
    footprint_path = os.path.join(PQ_DIR,
        f"Master({symbol})15m_Final_Footprint.parquet")

    # Fallback pattern if named differently
    if not os.path.exists(summary_path):
        summary_path = os.path.join(PQ_DIR,
            f"Master({symbol})15m_Final_Summary_{symbol}.parquet")
        footprint_path = os.path.join(PQ_DIR,
            f"Master({symbol})15m_Final_Footprint_{symbol}.parquet")

```

```
summary_path = os.path.join(POC_DIR, 1, (symbol) + sm_summary.parquet )
```

```
if not os.path.exists(summary_path):  
    # Fallback to compiled load_asset if summary file cannot be found  
    return compiled_module.load_asset(symbol)
```

```
df_s = pd.read_parquet(summary_path)
```

```
# Clean and parse TimeStamp column to ts_key  
ts_col_s = 'TimeStamp' if 'TimeStamp' in df_s.columns else 'Timestamp'  
df_s['ts_key'] = pd.to_datetime(df_s[ts_col_s].astype(str).str.replace(' IST', '',  
regex=False), errors='coerce')
```

```
if os.path.exists(footprint_path):  
    df_f = pd.read_parquet(footprint_path)
```

```
# Clean and parse Timestamp column in footprint to ts_key  
ts_col_f = 'TimeStamp' if 'TimeStamp' in df_f.columns else 'Timestamp'  
df_f['ts_key'] = pd.to_datetime(df_f[ts_col_f].astype(str).str.replace(' IST', '',  
regex=False), errors='coerce')
```

```
# Drop overlapping columns to prevent suffix duplication  
cols_to_drop = [c for c in ['Symbol', 'POC Price', 'Candle #', 'Timestamp',  
'TimeStamp', 'time'] if c in df_f.columns]  
df_f_clean = df_f.drop(columns=cols_to_drop, errors='ignore')
```

```
# Perform 1-to-1 inner merge on ts_key  
df = pd.merge(df_s, df_f_clean, on='ts_key', how='inner')  
else:  
    df = df_s
```

```
df = df.rename(columns={'ts_key': 'ts'})  
df = df.sort_values('ts').reset_index(drop=True)
```

```
# Drop time column because it is mostly NaN  
if 'time' in df.columns:  
    df = df.drop(columns=['time'])
```

```
# Convert columns (excluding Symbol and ts) to numeric
```

```
for c in df.columns:  
    if c not in ['Symbol', 'ts']:  
        df[c] = pd.to_numeric(df[c], errors='coerce')
```

```
df = df.set_index('ts')  
return df
```

```
def custom_prep(df, btc_ref):  
    if btc_ref is not None:  
        df = df.join(btc_ref, how='left')
```

```
df['atr'] = (df['High'] - df['Low']).rolling(14, min_periods=1).mean()  
df['cvd_delta'] = df['CVD'].diff(5)
```

```
if 'btc_CVD' in df.columns:  
    df['btc_cvd_mom'] = df['btc_CVD'].diff(2)  
else:  
    df['btc_cvd_mom'] = 0.0
```

```
df['ema_fast'] = df['Close'].ewm(span=200, min_periods=1).mean()  
df['ema_slow'] = df['Close'].ewm(span=800, min_periods=1).mean()  
df['macro_score'] = (df['ema_fast'] - df['ema_slow']) / df['atr'].replace(0, 1e-10)  
df['macro'] = np.where(df['macro_score'] > 0.5, 1, np.where(df['macro_score'] < -0.5,  
-1, 0))
```

```
feats = ['macro']
```

```
for k in [4, 10, 20]:  
    df['fz_cvd(k)'] = zscore(df['CVD'], k)  
    if 'btc_CVD' in df.columns:  
        df['fz_btc(k)'] = zscore(df['btc_CVD'], k)  
    else:  
        df['fz_btc(k)'] = 0.0  
    feats.extend(['fz_cvd(k)', 'fz_btc(k)'])
```

```
df['vol_regime'] = zscore(df['atr'], 100)  
feats.extend(['cvd_delta', 'btc_cvd_mom', 'vol_regime'])
```

```
if 'Agg. OI' in df.columns:  
    df['z_oi'] = zscore(pd.to_numeric(df['Agg. OI'], errors='coerce').fillna(0), 100)  
else:  
    df['z_oi'] = 0.0  
feats.append('z_oi')
```

```
if 'Long/Short Ratio (Account)' in df.columns:  
    df['z_ls'] = zscore(pd.to_numeric(df['Long/Short Ratio (Account)'],  
errors='coerce').fillna(0), 100)  
else:  
    df['z_ls'] = 0.0  
feats.append('z_ls')
```

```
if 'Agg. Funding Rate' in df.columns:  
    df['funding'] = pd.to_numeric(df['Agg. Funding Rate'], errors='coerce').fillna(0)  
else:  
    df['funding'] = 0.0  
feats.append('funding')
```

```
if 'Agg. Liq Long' in df.columns:  
    df['liq_long_5'] = pd.to_numeric(df['Agg. Liq Long'], errors='coerce').fillna(0).rolling(5,  
min_periods=1).sum()  
else:  
    df['liq_long_5'] = 0.0  
feats.append('liq_long_5')
```

```
if 'Agg. Liq Short' in df.columns:  
    df['liq_short_5'] = pd.to_numeric(df['Agg. Liq Short'],  
errors='coerce').fillna(0).rolling(5, min_periods=1).sum()
```

```

errors = ColoredPandasDroptailError, min_periods=1, rolling)
else:
    df['liq_short_5'] = 0.0
feats.append('liq_short_5')

df[feats] = df[feats].fillna(0)

# --- ENHANCED FEATURES FROM ROADMAP ---
df['ATR'] = df['atr'] # copy to match case in enhancements file

# Import the drop-in enhancement functions
# (Since we are local to alpha_squeezer_v17, we add _base directly)
_base = os.path.dirname(os.path.abspath(__file__))
if _base not in sys.path:
    sys.path.insert(0, _base)

from alpha_squeezer_v17_enhancements import (
    add_absorption_features,
    add_depth_imbalance_features,
    add_velocity_features,
    add_regime_features
)

df = add_absorption_features(df)
df = add_depth_imbalance_features(df)
df = add_velocity_features(df)
df = add_regime_features(df)

# Numeric conversions/replacements for categories to keep it safe for LGBM
if 'regime_state' in df.columns:
    df['regime_state'] = np.where(df['regime_score'] < -0.5, 0,
    np.where(df['regime_score'] > 0.5, 2, 1))

new_feats = [
    'abs_ratio_long', 'abs_ratio_short', 'absorption_flag_long', 'absorption_flag_short',
    'whale_z', 'oi_diff_std_3', 'poc_stability',
    'depth_imbalance_z', 'avg_trade_size_bid_z', 'avg_trade_size_ask_z',
    'poc_offset_atr', 'poc_location', 'cvd_divergence',
    'trade_velocity', 'trade_accel', 'delta_intensity', 'delta_intensity_roc',
    'funding_accel', 'oi_velocity', 'liq_velocity_long', 'liq_velocity_short',
    'regime_score'
]

# Check if footprint columns exist in the DataFrame before adding new features
# If footprint file is not present, we will skip these to avoid KeyErrors
existing_new_feats = [f for f in new_feats if f in df.columns]

if existing_new_feats:
    df[existing_new_feats] = df[existing_new_feats].fillna(0)

ga_selected = ['z_cvd_4', 'z_btc_4', 'z_btc_10', 'z_cvd_20', 'z_btc_20', 'btc_cvd_mom',
    'z_oi', 'z_ls', 'liq_long_5']
feats = [f for f in ga_selected if f in feats] + existing_new_feats

# Safely replace infs with NaN to prevent XGBoost crashes
df.replace([np.inf, -np.inf], np.nan, inplace=True)

# Clip extremely large values to prevent float32 overflow during XGBoost training
if existing_new_feats:
    for f in feats:
        df[f] = df[f].clip(lower=-1e8, upper=1e8)

close_1h = df['Close'].rolling(4, min_periods=1).mean()
ema_1h_50 = close_1h.ewm(span=50, min_periods=1).mean()
ema_1h_200 = close_1h.ewm(span=200, min_periods=1).mean()
atr_1h = (df['High'].rolling(4).max() - df['Low'].rolling(4).min()).rolling(14,
    min_periods=1).mean()
df['macro_1h_score'] = (ema_1h_50 - ema_1h_200) / atr_1h.replace(0, 1e-10)
df['macro_1h'] = np.where(df['macro_1h_score'] > 0.3, 1,
    np.where(df['macro_1h_score'] < -0.3, -1, 0))

df['vol_regime_gate'] = np.where(df['vol_regime'] < -1.0, 0, 1)

return df, feats

# Override the compiled functions
load_asset = custom_load_asset
compiled_module.load_asset = custom_load_asset

prep = custom_prep
compiled_module.prep = custom_prep

## 3. Engine_1/alpha_squeezer_v17/alpha_squeezer_v17_strategy.py python
"""
Alpha Squeezer V17 — Live Strategy for Engine 3
=====
Walk-Forward LightGBM Meta-Labeled strategy with Zeno Risk Sizing
and Staged Trailing Stop.

Integrates with Engine 3 via BaseTrendStrategy.evaluate_trigger().

Features:
- 15m timeframe, fetches live klines via Binance API
- CVD z-score features at windows 4/10/20
- BTC cross-asset CVD momentum
- 800-bar EMA macro trend filter
- LightGBM models retrained weekly via model_trainer.py
- Zeno Risk Sizing: risk = (MAX_DD_LIMIT - current_dd) / ZENO_DENOM
- Staged Trailing: 0.5R trail activates after +2.5R
"""

import os
import sys
import time
import threading
import numpy as np

```

```

import sys
import pandas as pd

# Ensure Core is importable
_CORE_DIR = os.path.abspath(os.path.join(os.path.dirname(file), "..", "Core"))
if _CORE_DIR not in sys.path:
    sys.path.insert(0, _CORE_DIR)

from base_trend_strategy import BaseTrendStrategy
from config import get_param
from meta_labeler import get_meta_label

# Try importing LightGBM (required for predictions)
try:
    import lightgbm as lgb
    HAS_LGB = True
except ImportError:
    HAS_LGB = False

# ——— DEFAULTS (overridable via Core/config.py)


---


DEFAULT_CONFIDENCE = 0.5261 # MYTHOS Phase 1: GA-optimized confidence threshold
DEFAULT_TP_MULT = 3.5 # ATR multiples
DEFAULT_SL_MULT = 1.0
DEFAULT_TRAIL_ACT_R = 2.5 # Activate trailing after +2.5R
DEFAULT_TRAIL_BUFFER = 0.5 # Trail distance in ATR multiples
DEFAULT_MAX_BARS = 96 # Max hold time (15m bars = 24 hours)
DEFAULT_COOLDOWN = 900 # 15 min cooldown
DEFAULT_KLINE_LIMIT = 1000 # Bars to fetch for feature engineering (needs 800 for EMA macro)

# Zeno Risk Sizing (used as metadata in signal dict)
MAX_DD_LIMIT = 300.0
ZENO_DENOM = 4.0
RISK_CAP = 120.0

# Model directory (resolve correctly whether running standalone or bundled in root)
_base = os.path.dirname(file)
if os.path.exists(os.path.join(_base, "models")):
    DEFAULT_MODEL_DIR = os.path.join(_base, "models")
else:
    DEFAULT_MODEL_DIR = os.path.join(_base, "Strategies", "alpha_squeezer_v17", "models")

class AlphaSqueezerV17(BaseTrendStrategy):
    """
    Walk-Forward LightGBM Meta-Labeled strategy for Engine 3.

    Each instance is confined to a single symbol (e.g., XRPUSD).
    Multiple instances are created in Engine_3.py for multi-asset coverage.
    """

    def init(self, feed, symbol: str, model_dir: str = None):
        super().init(feed, f"AlphaSqueezer_V17[{symbol[:3]}]")
        self.symbol = symbol
        self.tf_min = 15 # 15-minute timeframe
        self.tf = "15m"
        self.indicators = {} # Required by Engine 3 for dashboard display

        # Model storage
        self.model_dir = model_dir or DEFAULT_MODEL_DIR
        self.model_long = None
        self.model_short = None
        self._model_lock = threading.Lock()
        self._last_model_load = 0
        self._load_models()

        # Feature cache (avoid redundant API calls within same 15m bar)
        self._feature_cache = {}
        self._feature_cache_ts = 0

        # BTC reference cache
        self._btc_cache = None
        self._btc_cache_ts = 0

        # Load MYTHOS WFO Params
        self.wfo_params = {}
        _base = os.path.dirname(file)
        if os.path.exists(os.path.join(_base, "final_wfo_params.json")):
            wfo_path = os.path.join(_base, "final_wfo_params.json")
        else:
            wfo_path = os.path.join(_base, "Strategies", "alpha_squeezer_v17", "final_wfo_params.json")
        if os.path.exists(wfo_path):
            import json
            try:
                with open(wfo_path, 'r') as f:
                    all_params = json.load(f)
            except Exception as e:
                print(f"[{self.strategy_name}] Error loading WFO params: {e}")
            self.wfo_params = all_params.get(self.symbol, {})

        # ——— MODEL MANAGEMENT
        def _load_models(self):
            """Load pre-trained LightGBM boosters from disk."""
            if not HAS_LGB:
                print(f"[{self.strategy_name}] WARNING: lightgbm not installed. Running in signal-passthrough mode.")
                return

            long_path = os.path.join(self.model_dir, f"{self.symbol}_long.txt")
            short_path = os.path.join(self.model_dir, f"{self.symbol}_short.txt")

```

```

with self._model_lock:
    if os.path.exists(long_path):
        self.model_long = lgb.Booster(model_file=long_path)
        print(f"[{self.strategy_name}] Loaded LONG model from {long_path}")
    else:
        print(f"[{self.strategy_name}] No LONG model found at {long_path} — will use
feature-only signals.")

    if os.path.exists(short_path):
        self.model_short = lgb.Booster(model_file=short_path)
        print(f"[{self.strategy_name}] Loaded SHORT model from {short_path}")
    else:
        print(f"[{self.strategy_name}] No SHORT model found at {short_path} — will use
feature-only signals.")

self._last_model_load = time.time()

def _maybe_reload_models(self):
    """Check if models should be reloaded (every 6 hours). Check is inside lock to
prevent TOCTOU race."""
    with self._model_lock:
        if time.time() - self._last_model_load > 21600: # 6 hours
            # Release lock so _load_models can re-acquire it cleanly
            pass
            # If reload needed, _load_models acquires its own lock
        if time.time() - self._last_model_load > 21600:
            self._load_models()

# ——— FEATURE ENGINEERING

@staticmethod
def _zscore(series, window=100):
    """Rolling z-score."""
    mean = series.rolling(window, min_periods=1).mean()
    std = series.rolling(window, min_periods=1).std().replace(0, 1e-10)
    return (series - mean) / std

def _fetch_btc_reference(self):
    """Fetch BTC 15m klines for cross-asset features."""
    now = time.time()
    if self._btc_cache is not None and (now - self._btc_cache_ts) < 60:
        return self._btc_cache

    klines = self.get_raw_klines("BTCUSDT", self.tf_min, limit=DEFAULT_KLINE_LIMIT)
    if not klines or len(klines) < 30:
        return None

    btc_df = pd.DataFrame(klines, columns=[
        'open_time', 'Open', 'High', 'Low', 'Close', 'Volume',
        'close_time', 'quote_vol', 'trades', 'taker_buy_base',
        'taker_buy_quote', 'ignore'
    ])
    for c in ['Open', 'High', 'Low', 'Close', 'Volume', 'taker_buy_base']:
        btc_df[c] = pd.to_numeric(btc_df[c], errors='coerce')

    # Compute BTC CVD
    btc_df['btc_CVD'] = (2 * btc_df['taker_buy_base'] - btc_df['Volume']).cumsum()
    btc_df['btc_Close'] = btc_df['Close']
    # Keep open_time for timestamp-based alignment in _compute_features
    btc_df['open_time'] = pd.to_numeric(btc_df['open_time'], errors='coerce')

    self._btc_cache = btc_df[['open_time', 'btc_Close', 'btc_CVD']].copy()
    self._btc_cache_ts = now
    return self._btc_cache

def _compute_features(self, klines):
    """
    Compute the exact same feature set as the V17 backtester.
    Returns (feature_df, feature_columns) or (None, None).
    """
    if not klines or len(klines) < 60:
        return None, None

    df = pd.DataFrame(klines, columns=[
        'open_time', 'Open', 'High', 'Low', 'Close', 'Volume',
        'close_time', 'quote_vol', 'trades', 'taker_buy_base',
        'taker_buy_quote', 'ignore'
    ])
    for c in ['Open', 'High', 'Low', 'Close', 'Volume', 'taker_buy_base']:
        df[c] = pd.to_numeric(df[c], errors='coerce')
    df['open_time'] = pd.to_numeric(df['open_time'], errors='coerce')

    # CVD from taker buy/sell
    df['CVD'] = (2 * df['taker_buy_base'] - df['Volume']).cumsum()

    # ATR
    df['atr'] = (df['High'] - df['Low']).rolling(14, min_periods=1).mean()

    # Macro trend (800-bar EMA)
    df['ema_slow'] = df['Close'].ewm(span=800, min_periods=1).mean()
    df['macro'] = np.where(df['Close'] > df['ema_slow'], 1, -1)

    # BTC cross-asset features — align open_time to prevent off-by-one at 15m
    boundaries

    btc_ref = self._fetch_btc_reference()
    if btc_ref is not None and len(btc_ref) > 0:
        # Build a lookup from open_time to btc_CVD value
        df['btc_CVD'] = df['open_time'].map(
            btc_ref.set_index('open_time')['btc_CVD']
        ).ffill().bfill().fillna(0.0)
    else:
        df['btc_CVD'] = 0.0

```

```

# Feature columns (must match training exactly)
feat_cols = ['macro']
for k in [4, 10, 20]:
    df['z_cvd[k]'] = self.zscore(df['CVD'], k)
    df['z_btc[k]'] = self.zscore(df['btc_CVD'], k)
    feat_cols += ['z_cvd[k]', 'z_btc[k]']

df['cvd_delta'] = df['CVD'].diff(5)
df['btc_cvd_mom'] = df['btc_CVD'].diff(2)
df['vol_regime'] = self._zscore(df['atr'], 100)
feat_cols += ['cvd_delta', 'btc_cvd_mom', 'vol_regime']

# --- Coinglass Features from Live Feed ---
# We need to extract the Coinglass history to compute Z-scores.
hist_data = self.feed.history.get(self.symbol, {})

def _get_cg_series(key):
    raw_tuples = hist_data.get(key, [])

    if not raw_tuples:
        return pd.Series()

    # Extract times and convert to DatetimeIndex
    times = [pd.to_datetime(t, unit='s') for t, v in raw_tuples]
    vals = [v for t, v in raw_tuples]

    s = pd.Series(vals, index=times).replace('N/A', np.nan).replace("", np.nan)

    # Handle liquidations special case (format is usually "LongLiq ShortLiq")
    if key == 'liquidations':
        return s

    # Remove M, K suffixes if present
    if s.dtype == object:
        s = s.astype(str).str.replace('K', 'e3', case=False)
        s = s.str.replace('M', 'e6', case=False)
        s = s.str.replace('B', 'e9', case=False)

    s = pd.to_numeric(s, errors='coerce').ffill().fillna(0.0)

    # Resample to 15m (using the last value in each 15m bar) and forward fill
    s_15m = s.resample("15min").last().ffill().fillna(0.0)
    return s_15m

# 1. Open Interest Z-score (feed stores 'oi')
oi_s = _get_cg_series('oi')
if len(oi_s) > 0:
    mean = oi_s.rolling(100, min_periods=1).mean().iloc[-1]
    std = oi_s.rolling(100, min_periods=1).std().replace(0, 1e-10).iloc[-1]
    z_oi_val = (oi_s.iloc[-1] - mean) / std
else:
    z_oi_val = 0.0

# 2. L/S Ratio Z-score
ls_s = _get_cg_series('ls_ratio')
if len(ls_s) > 0:
    mean = ls_s.rolling(100, min_periods=1).mean().iloc[-1]
    std = ls_s.rolling(100, min_periods=1).std().replace(0, 1e-10).iloc[-1]
    z_ls_val = (ls_s.iloc[-1] - mean) / std
else:
    z_ls_val = 0.0

# 3. Funding Rate (feed stores as 'funding')
fund_s = _get_cg_series('funding')
funding_val = fund_s.iloc[-1] if len(fund_s) > 0 else 0.0

# 4. Liquidations (feed stores as separate 'liquidations_long_15m' and
'liquidations_short_15m')
liq_long_s = _get_cg_series('liquidations_long_15m')
liq_short_s = _get_cg_series('liquidations_short_15m')
liq_long_sum = float(liq_long_s.tail(5).sum()) if len(liq_long_s) > 0 else 0.0
liq_short_sum = float(liq_short_s.tail(5).sum()) if len(liq_short_s) > 0 else 0.0

# Broadcast the latest values to the dataframe (since we only care about df.iloc[-1]
for prediction)
df['z_oi'] = z_oi_val if not np.isnan(z_oi_val) else 0.0
df['z_ls'] = z_ls_val if not np.isnan(z_ls_val) else 0.0
df['funding'] = funding_val if not np.isnan(funding_val) else 0.0
df['liq_long_5'] = liq_long_sum
df['liq_short_5'] = liq_short_sum

feats = ['z_oi', 'z_ls', 'funding', 'liq_long_5', 'liq_short_5']
feat_cols += feats

df[feat_cols] = df[feat_cols].fillna(0)

# MYTHOS Phase 1: GA-selected feature subset (9 of 15)
# Pruned: macro, z_cvd_10, cvd_delta, vol_regime, funding, liq_short_5
ga_selected = [
    'z_cvd_4', 'z_btc_4', 'z_btc_10', 'z_cvd_20', 'z_btc_20',
    'btc_cvd_mom', 'z_oi', 'z_ls', 'liq_long_5'
]
feat_cols = [f for f in ga_selected if f in feat_cols]

return df, feat_cols

# ——— EVALUATE TRIGGER (Engine 3 interface) ———

def evaluate_trigger(self, symbol: str):
    """
    Called by Engine 3 main loop every ~5 seconds.
    Returns (signal_dict, message) or (None, message).
    """
    if symbol != self.symbol:
        return None, "Symbol mismatch"

```

```

# Get live price from feed
fd = self.feed.get_latest().get(symbol, {})
live_price = fd.get('price', 0.0)
if live_price <= 0:
    return None, "No live price"

# Periodically check for model updates
self._maybe_reload_models()

# Fetch 15m klines
klines = self.get_raw_klines(symbol, self.tf_min, limit=DEFAULT_KLINE_LIMIT)
if not klines or len(klines) < 60:
    return None, f"Warming up ({len(klines) if klines else 0}/{DEFAULT_KLINE_LIMIT} bars)"

# Compute features
df, feat_cols = self._compute_features(klines)
if df is None:
    return None, "Feature computation failed"

# Get latest bar's features
latest = df.iloc[-1]
atr = latest['atr']
macro = int(latest['macro'])

if pd.isna(atr) or atr <= 0:
    return None, "ATR invalid"

# Store indicators for Engine 3 dashboard
self.indicators(symbol) = {
    'atr': float(atr),
    'macro': macro,
    'ema_slow': float(latest['ema_slow']),
    'cvd_delta': float(latest['cvd_delta']),
    'vol_regime': float(latest['vol_regime']),
}

# Config overrides (WFO > user config > default)
confidence = self.wfo_params.get('confidence', self.get_param(symbol,
    'confidence_threshold', DEFAULT_CONFIDENCE))
tp_mult = self.wfo_params.get('tp_mult', self.get_param(symbol, 'tp_mult',
    DEFAULT_TP_MULT))
sl_mult = self.wfo_params.get('sl_mult', self.get_param(symbol, 'sl_mult',
    DEFAULT_SL_MULT))
trail_act = self.wfo_params.get('trail_act', self.get_param(symbol, 'trail_activation_r',
    DEFAULT_TRAIL_ACT_R))
trail_buf = self.get_param(symbol, 'buffer_atr', DEFAULT_TRAIL_BUFFER)

vol_limit = self.wfo_params.get('vol_limit', 1.0) # Default 100% so it doesn't block if
missing

# Phase 2: Volatility Regime Gate
atr_pct = atr / live_price
if atr_pct > vol_limit:
    return None, f"Regime Filter (ATR% {atr_pct:4f} > Limit {vol_limit:4f})"

# — ML Prediction

prob_long = 0.0
prob_short = 0.0
features_row = df[feat_cols].iloc[-1] # Single-row DataFrame

with self._model_lock:
    if self.model_long is not None:
        prob_long = float(self.model_long.predict(features_row)[0])
    if self.model_short is not None:
        prob_short = float(self.model_short.predict(features_row)[0])

# If no models loaded, use a feature-only heuristic
if self.model_long is None and self.model_short is None:
    # Fallback: use CVD z-score + macro alignment as a basic signal
    z_cvd_10 = float(latest.get('z_cvd_10', 0))
    if macro == 1 and z_cvd_10 > 1.5:
        prob_long = 0.55
    elif macro == -1 and z_cvd_10 < -1.5:
        prob_short = 0.55

# Update indicators with probabilities
self.indicators(symbol)['prob_long'] = prob_long
self.indicators(symbol)['prob_short'] = prob_short

# — Signal Generation

direction = 0
prob = 0.0

if prob_long > confidence and prob_long > prob_short and macro == 1:
    direction = 1
    prob = prob_long
elif prob_short > confidence and prob_short > prob_long and macro == -1:
    direction = -1
    prob = prob_short
else:
    return None, f"Searching (L={prob_long:3f} S={prob_short:3f} M={macro})"

# — Meta Label Veto

dir_str = "LONG" if direction == 1 else "SHORT"

# Build context payload matching the Prompt Optimizer template
recent_liqs = float(latest.get('liq_sl', 0.0)) if direction == 1 else
float(latest.get('liq_sl', 0.0))
vol_z = float(latest.get('vol_regime', 0.0))
regime_str = "Expansion" if vol_z > 1.0 else "Contraction" if vol_z < -1.0 else "Neutral"

context_str = f"Market Regime: {regime_str}\nAsset: {self.symbol}\nVolatility Z-
Score: {vol_z:2f}\nDirection: {dir_str}\nRecent Liquidations: {recent_liqs:2f}"

```

```

meta_verdict = get_meta_label(context_str)
if meta_verdict == "UNSAFE":
    return None, f"VEIO: Meta-Labeler flagged {dir_str} signal as UNSAFE."

# ——— Compute SL/TP ———

entry_price = live_price
if direction == 1:
    sl = entry_price - sl_mult * atr
    tp = entry_price + tp_mult * atr
else:
    sl = entry_price + sl_mult * atr
    tp = entry_price - tp_mult * atr

# ——— Zeno Risk Sizing (communicated via signal dict) ———
# Engine 3's TradeTracker will use risk_pct for position sizing.
# We compute the Zeno-optimal risk as a fraction of current capital.
# The actual $ amount is calculated by TradeTracker using current_capital.
zeno_risk_pct = 0.005 # Adjusted to match Zeno Backtest (0.5% = $25 on
# Invalid equation (MAX_DD_LIMIT * DENOM = (ZENO_DENOM),
CAP=${RISK_CAP})
...

```

Assuming the weird backticks/asterisks/HTML breaks are paste artifacts, here is the blunt review of the intended code.

Bottom line

I would not run this live with money yet.

There are at least 2 clear P0s and a pile of P1s that can either prevent trades, poison features, or make PnL/risk accounting wrong enough to mislead you.

Executive summary by severity

P0: must fix before live

1. `sl_mult` / `tp_mult` are undefined in `Engine_1.py`
 - In `LiveStrategyPredictor.on_tick_update()`:

```

sl = snap.price - sl_mult * atr_val ...
tp = snap.price + tp_mult * atr_val ...

```

but `sl_mult` and `tp_mult` are never defined in that scope.

- Result:
 - the first time a signal arms, entry logic throws
 - the exception is caught by the broad `except Exception as e`
 - you log a prediction error and the trade never opens
- This is a hard live failure.

2. `combine_seeding_files()` appears to generate sheet names that `load_history_from_disk()` will never load

- You do:

```
symbol = os.path.basename(f).replace("*seed_history.xlsx", "")
```

- That `replace()` uses a literal `*`, not a wildcard.
For `BTCUSDOT_seed_history.xlsx`, `symbol` stays `BTCUSDOT_seed_history.xlsx`.
- Then:

```
target_ws = combined_wb.create_sheet(title=symbol[:31])
```

- Later `load_history_from_disk()` does:

```

sym = sheetname
if sym not in self.symbols:
    continue

```

- So your combined workbook sheets are likely named `BTCUSDOT_seed_history.xlsx`, not `BTCUSDOT`, and get skipped.
- Result: history may never actually load from the combined workbook you generate.

If those two are real in the source and not paste corruption, they alone are enough to block production.

P1: serious logic / fidelity problems

1. Seeded history timestamps are synthetic, not actual candle timestamps

- In `seed_symbol()`:

```
"open_time": int(time.time() - step * 900)
```

- That is not the chart candle's actual open time. It is just "now minus N*15m".
- Consequences:
 - BTC and alt histories seeded at different wall-clock moments will not align exactly
 - cross-asset join in `prep()` is done on exact timestamp index equality
 - seeded BTC features can fail to join seeded alt features
 - `z_btc_*` and `btc_cvd_mom` can degrade to NaN/0 for seeded history
- This is a big fidelity break.

2. Your seeding code only collects 120 candles, not the 2,000+ warm start you described

seed_symbol():

```
candles = collections.deque(maxlen=120)
for step in range(120):
```

- So unless `combined_seed_history.xlsx` is built elsewhere, this code does not create an 800-EMA-grade warm start.
 - If the workbook truly has 2,000+ rows per symbol, fine. But the code shown does not produce that.
3. You start predicting after only 20 bars if seeding is absent/broken
- `on_tick_update()`:

```
if len(history) < 20:
    return snap
```

- That is nowhere near enough for:
 - EMA 800 regime
 - vol zscore 100
 - stable OI/LS zscores
 - BTC cross-asset structure
- If seeding silently fails, you will still start producing predictions after 20 bars, which is bad.

4. Feature pipeline is only partially faithful to training

- Good news: `Engine_1.py` calls `prep()` from `unified_backtest.py`, which is the right idea.
- Bad news: the inputs into `prep()` are not faithful enough:
 - `POC Price` is set to close price as a proxy

```
mapped_df['POC Price'] = df['close'].astype(float) # proxy
```

If the enhancement features use real POC-derived structure, this is wrong.

- live footprint POC (`fp_poc`) is not actually fed into `prep()`
- bid/ask / trade count / whale fields are scraped approximations, not guaranteed identical to parquet semantics
- If the model uses enhancement features derived from these columns, you are feeding it numerically valid but semantically wrong features, which is worse than a crash.

5. Liquidation sign handling is inconsistent between seed and live

- In seeding:

```
"liq_short": abs(parse_float(...))
```

- In live updates:

```
liq_short=parse_float(d.get("liquidations_short", 0.0))
```

- The JS parser appears to classify short liquidations using a minus sign. So live `liq_short` may be negative, while seeded `liq_short` is positive.
- Then `prep()` does rolling sums of `Agg. Liq Short`.
- That means the feature distribution changes at the handoff from seed → live.
- This is a real feature drift bug.

6. Funding-rate scaling may be off by 100x

- JS extraction does:

```
data.funding_rate = (raw * 100).toFixed(4) + '%'
```

- Then `parse_float()` strips `%` and returns the numeric value.
- So a raw funding of `0.0001` becomes `0.01` feature.
- If training parquet stored funding as raw decimal rather than percentage points, live inference is off by 100x.
- You need to confirm training units.

7. Mixed-time snapshot problem: you can predict on half-updated candles

- `store.update()` is called from:
 - DOM polling
 - HTTP response interception
 - Binance footprint polling
- Each update can trigger `on_tick_update()`.
- That means a prediction can run after:
 - price updated
 - but funding still old
 - or OI new but CVD old
 - or footprint old but DOM new
- This is not a thread race, it is a temporal consistency race.
- You need batching/versioning, not per-field immediate inference.

8. PnL accounting is wrong because you use current capital at exit, not entry risk / stored units

- `trigger_entry()` stores:

```
"units": (self.current_capital * 0.005) / abs(entry_price - sl)
```

- But `check_exits()` computes realized PnL as:

```
pnl_usd = r_mult * (self.current_capital * 0.005)
```

- That uses capital at exit time, not capital at entry time.
 - If other trades closed in between, PnL for this trade changes retroactively.
 - Also, `units` is stored but not used.
 - This corrupts history, capital curve, and win/loss stats.
9. No hard drawdown stop
- You mention Zeno sizing / max DD concept in Engine 3 strategy docs.
 - Engine 1 tracker just uses fixed `0.5%` risk and max 4 positions.
 - There is no account-level kill switch.
10. `atr == 0` is not rejected at entry
- If `atr_val == 0`:
 - SL/TP collapse to entry
 - units become `0.0`
 - trade can still be inserted
 - That should be blocked before entry.
11. Seed/live duplicate current bucket risk
- If your last seeded bar is the current stop forming 15m bucket, and the first live tick builds `current_candle` with the same `open_time`, you can end up with duplicate bucket rows in `combined`.
 - There is no dedupe on `open_time`.
 - That distorts rolling features.
12. Missing/failed parses are silently converted to 0.0
- `parse_float()` returns `0.0` on any garbage.
 - That means:
 - missing
 - parse failure
 - selector change
 - broken scrape
 - all become "real zeros".
 - This is especially bad because later you also treat zero as missing during fill in seeding.
13. Your seeding fill logic rewrites legitimate zeros
- `dump_xlsx()` does forward-fill/back-fill on zeros for many fields:

```
if val != 0.0:
    last_val = val
else:
    r[field] = last_val
```

- Zero can be a legitimate value for:
 - funding near zero
 - trade counts
 - whale index
 - bid/ask imbalance moments
 - You are inventing data.
14. Relative-path handling is inconsistent
- Trade log path is safely anchored via `__file__`.
 - Seeding paths are often relative:
 - `"Engine_1/Seeding/..."`
 - Loader path is anchored:
 - `base_dir/.../Seeding/combined_seed_history.xlsx`
 - If cwd differs, you can write seeds in one place and load from another.
 - This is a classic "works in IDE, breaks in production launcher" bug.

P2: robustness / maintainability concerns

1. Too many `except: pass` or silent suppressions
- Critical ones:
 - `load_history()`
 - `save_history()`
 - combine cleanup deletes
 - screenshot/debug writes
 - response handler
 - `save_history()` silently failing is especially bad because you can think trades are logged when they are not.
2. Model staleness check is buggy
- This loop:

```
for sym in list(self.models_long) + list(self.models_short):
    side = 'long' if sym in self.models_long else 'short'
```

- If a symbol has both models, the symbol appears twice and both passes resolve to `long`.
 - So you may never check short-model age correctly.
3. Live renderer writes JSON to disk every refresh cycle
- In `renderer_loop()`, every render writes `snapshot_debug.json`.
 - That is needless synchronous I/O on the event loop.
4. Hard-coded credentials in source
- Operationally awful.
 - Separate from strategy logic, but still unacceptable for a live trading system.
5. Footprint POC is not a real POC
- `FootprintCandle.update()`:

```
self.volume_profile[bucket] = buy_vol + sell_vol
```

- REST klines give cumulative candle totals, not tick-by-tick per-price volume.
 - So:
 - this is not a true volume profile
 - `poc` is not a true POC
 - If you later use it as if it were real orderflow structure, you are fooling yourself.
6. `current_capital` is used as both accounting state and sizing base without trade-level freeze
- This makes analytics noisy and hard to trust.
7. Engine 1 and Engine 3 are too coupled and too divergent at the same time
- Coupled:
 - shared model files
 - shared config artifacts
 - shared strategy concept
 - Divergent:
 - Engine 1 uses `prep()` from backtest
 - Engine 3 reimplements feature engineering differently
 - Engine 1 uses macro neutral band
 - Engine 3 simplified logic differs
 - This is the worst combination: shared artifacts with non-shared logic.

Answers to your 9 requested review areas

1) Signal logic correctness

Macro gate: `macro >= 1 / macro <= -1`

- In Engine 1, with current `prep()`, `macro` is in `{-1, 0, 1}`.
- Therefore:
 - `macro >= 1 == macro == 1`
 - `macro <= -1 == macro == -1`
- So this specific comparison is fine. It is not too loose.

The real issue is not `>=` vs `==`

The real issue is whether the macro gate itself is parity-matched with training.

In `custom_prep()`:

```
macro_score = (ema_fast - ema_slow) / atr
macro = 1 if > 0.5, -1 if < -0.5, else 0
```

That is a neutral-zone regime filter. Fine if:

- training labels/signals were generated under the same gate
- model calibration expected that filter at inference

Not fine if:

- the model was trained on all bars
- then you apply macro as an out-of-model veto only live

Confidence threshold

- Engine 1 uses `>=`
- Engine 3 strategy uses `>`
- That exact-threshold mismatch is minor, but it is still drift.

`vol_limit`

- You compute:

```
atr_pct = atr / price
```

- That is a fraction, not human percent.
- If configs are stored as `1.0` meaning "1%", you are wrong by 100x.
- If configs are stored as `0.01`, fine.
- Verify units.

My verdict

- `macro >= 1` itself is okay.
- The macro gate is probably too strict only if training did not use the same post-filter.
- The bigger issue is parity verification, not comparator choice.

2) Feature pipeline fidelity

Best part of the design

Using:

```
from unified_backtest import prep
```

inside Engine 1 is the right move. That is the strongest parity decision in the whole codebase.

But fidelity is still broken at the input layer

You are not feeding `prep()` the same-quality raw feature substrate as training.

Main problems:

1. Synthetic timestamps
 - breaks BTC join on seeded rows
2. POC proxy
 - `POC Price = close`
 - not equivalent to training footprint POC

- possibly 100x
4. Liquidation sign inconsistency
 - seeded positive, live potentially negative
5. Zero-as-missing coercion
 - parse failures become zeros
6. No explicit feature validation before predict
 - You should assert:
 - all expected feature columns exist
 - all are numeric
 - all finite or intentionally NaN

Feature order

This part is okay:

```
expected_feats = booster.feature_name()
X = last_row[expected_feats]
```

That protects ordering.

Recommendation

Before predict, do this:

```
missing = [c for c in expected_feats if c not in df_feat.columns]
if missing:
    raise ValueError(f'Missing features: {missing}')

X = df_feat.loc[df_feat.index[-1], expected_feats].astype("float32")
if not np.isfinite(X.fillna(0)).all():
    ...
```

3) Candle construction

What is correct

The OHLC update logic itself is fine:

- open = first tick in bucket
- high = max
- low = min
- close = latest

What is risky

1. Bucketing uses local wall clock, not source candle time

```
open_time = int(now // 900) * 900
```

Better to use exchange/chart candle timestamp if available.

2. Per-field updates trigger full prediction
 - you build candles from mixed freshness data
3. Duplicate bucket risk around startup
 - current live candle may duplicate last seeded bucket
4. Volume semantics are assumed
 - setting `candle["volume"] = snap.volume` is only correct if `snap.volume` is current bucket total, not incremental

Verdict

OHLC logic: okay.

Candle integrity across asynchronous inputs: **not okay**.

4) Seeding and warm start

Biggest issues

- generated combined workbook may not load at all
- seeded timestamps are fake
- code shown only seeds 120 bars per symbol
- duplicates/out-of-order rows are not explicitly deduped

Is 1200 maxlen sufficient for EMA 800?

Yes, 1200 is reasonable, but not "fully converged".

For EMA span = 800:

- $\alpha = 2 / (800 + 1) \approx 0.002497$

Approximate warm-up:

- ~800 bars: enough to stop being nonsense, but still immature
- ~1200 bars: about 95% effective memory capture
- ~1840 bars: about 99% effective memory capture

So:

- bare minimum to compute EMA800: technically 1 bar
- minimum to not be garbage: ~800 bars
- practical live minimum: ~1200 bars
- ideal: 1800-2000 bars

What I would do

Require:

- at least 1200 valid rows
- deduped by `open_time`
- sorted ascending

- latest row not equal to current live bucket
- BTC and basic thresholds signed on exact form boundaries

5) Concurrency and thread safety

Better than it looks

Because this is mostly single-threaded async, and `SnapshotStore.update()` calls predictor synchronously while holding the per-symbol async lock, you do not have true simultaneous mutation of one symbol's candle state.

Still not clean

Problems remain:

1. No atomic multi-field snapshot
 - different async sources update independently
 - prediction runs on partially refreshed state
2. TradeTracker lock is local, but some reads bypass ideal discipline
 - membership checks against `active_trades` outside explicit lock usage
3. Blocking disk I/O inside event loop
 - `save_history()`
 - JSON debug writes
 - print storms
4. Cross-symbol consistency is not enforced
 - a symbol prediction reads BTC reference state without coordinated versioning

Verdict

Not a catastrophic thread bug.

But temporal consistency is poor, which matters just as much.

6) Risk and position sizing

What's wrong

1. Undefined SL/TP multipliers in Engine 1 entry path
 - P0, already covered.
2. PnL math uses exit-time capital
 - wrong accounting
3. Stored `units` are never actually used for PnL
 - this is the biggest accounting smell in the tracker
4. No drawdown kill switch
 - absolutely should exist
5. No ATR-zero guard before entry
 - must reject

SL/TP formula

Conceptually correct:

- long: `sl = entry - sl_mult * atr`
- long: `tp = entry + tp_mult * atr`
- mirrored for short

But only if:

- `atr > 0`
- multipliers exist
- price valid
- `sl != entry`

Recommendation

Freeze trade economics at entry:

```
risk_usd = current_capital * 0.005
units = risk_usd / abs(entry - sl)
```

Then realized:

```
pnl_usd = units * (exit - entry) * direction - fees
```

Do not recompute from current capital later.

7) Edge cases and silent failures

Bad silent catches

These matter:

- `load_history()`
- `save_history()`
- `combine_seeding_files()` cleanup
- response parsing
- renderer debug dump

Worst coercion bug

`parseFloat()` returning `0.0` for all failures.

That means:

- missing data
- selector changes
- HTTP format changes
- broken scraping

all become "valid numeric zeros".

That is poison.

Recommendation

Use:

- `np.nan` for failed parse
- explicit imputation later
- metrics for missingness rate



8) Terminal display

Is `rich.live.Live` itself okay with async Playwright?

Yes, generally.

What is not okay

- synchronous JSON writes every refresh
- lots of `print()` from async tasks while `Live` is rendering
- heavy terminal formatting on the same event loop as scrape/predict

Risk

Not likely to fully block scraping on a good machine, but it will:

- jitter refresh
- add latency
- make debugging ugly
- potentially stall during I/O spikes

Recommendation

- remove per-frame disk dump
- lower render freq if needed
- use structured logging instead of naked prints during Live



9) Overall architecture

Conceptually

Separating:

- Engine 1 = signal generation / market-state inference
- Engine 3 = execution

is a sound idea.

In practice

Your current implementation is not a clean separation.

Why

1. Shared models/configs with divergent logic
 - Engine 1 uses backtest `prep()`
 - Engine 3 reimplements features differently
2. No single canonical inference module
 - this guarantees drift
3. Potential live artifact hot-swap
 - Engine 1 retrains at startup
 - Engine 3 may load same model files
 - no artifact versioning / pinning
4. Tracker/accounting logic differs by engine
 - already caused historical contamination once



Verdict

The architecture is fine in principle.

The implementation is too coupled operationally and too inconsistent logically.

What I would fix first, in order

First 24 hours

1. Fix `sl_mult` / `tp_mult` in Engine 1
2. Fix `combine_seeding_files()` symbol extraction
3. Anchor all file paths via `_file_`
4. Block prediction unless:
 - history loaded
 - at least 1200 rows
 - BTC aligned
5. Stop using synthetic seed timestamps
6. Make liquidation sign conventions consistent
7. Fix PnL accounting to use frozen entry unit



Next

1. Replace `parse_float(...)>@.0` with NaN-based parsing
2. Batch updates so prediction runs once per coherent snapshot, not per field patch
3. Remove fake `POC Price = close` if model uses POC-derived features
4. Add hard drawdown stop
5. Remove silent `except: pass` from critical persistence paths

Final verdict

This engine has one genuinely good design choice: reusing the training `prep()` inside live inference.

But around that, there are multiple serious flaws:

- one entry-path killer (`sl_mult` / `tp_mult`)
- one likely seed-loader killer
- several feature drift bugs
- bad accounting
- too much silent failure

So my blunt assessment is:

Testing and development.

The core idea is good. The current implementation is not production-safe.

If you fix the P0s and the top P1s, it becomes salvageable. Right now, it is still in "dangerous prototype" territory.

📄 + 🗨️

🔔 You've run out of free AI responses. [Upgrade Notion AI.](#)

Describe your own

+ 🔍 🗨️

🌐 GPT-5.6 Sol

↑